



Korra AI Agent Development

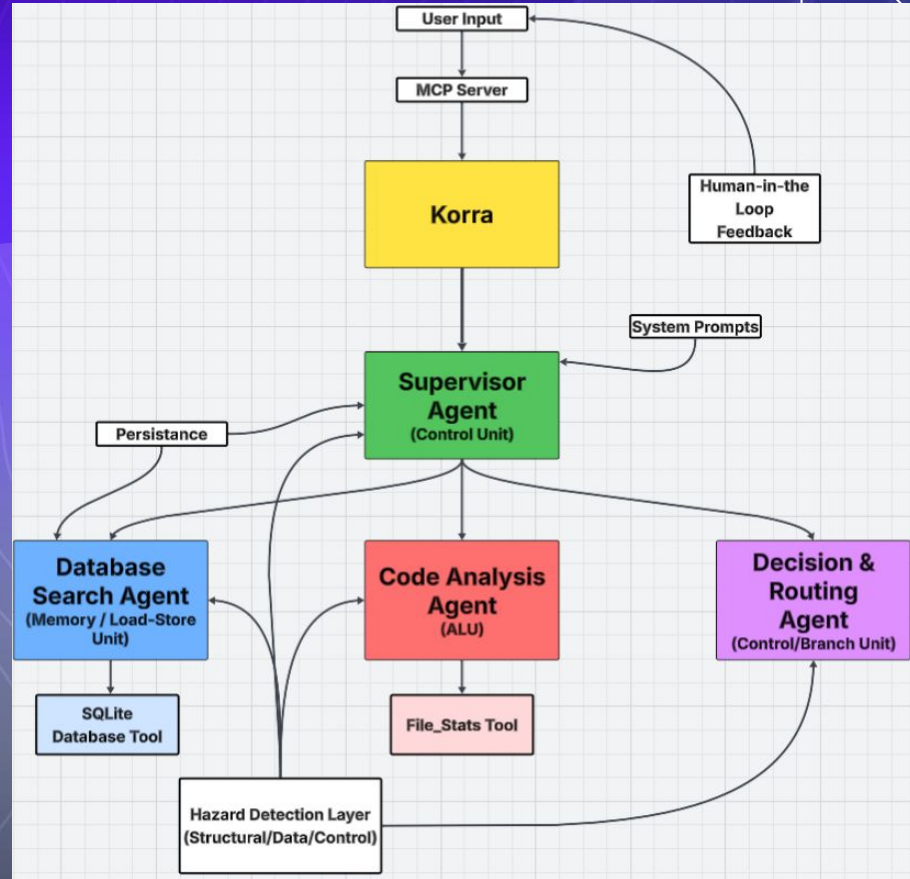
Diego Lopez
COMPE 475 - Microprocessors
Spring 2026

What is Korra?

- Educational AI agent built using LangGraph
- **Purpose:** To learn about computer architecture by building a working system
- Acts as a teaching tool by simulating how a processor executes tasks using specialized components
- It can be connected to microprocessor architecture by modeling:
 - Functional Units as Agents
 - Pipelining through the use of multi-agent workflow
 - Hazard Detection through managing conflicts between tasks

Korra's Architecture

- Supervisor Routes Requests to Specialized Agents
- Agents Form a Pipeline
- Hazard Detection Ensures Correct Execution



What We Built

Korra was able to progress from a simple chatbot to a multi-agent pipeline throughout the semester

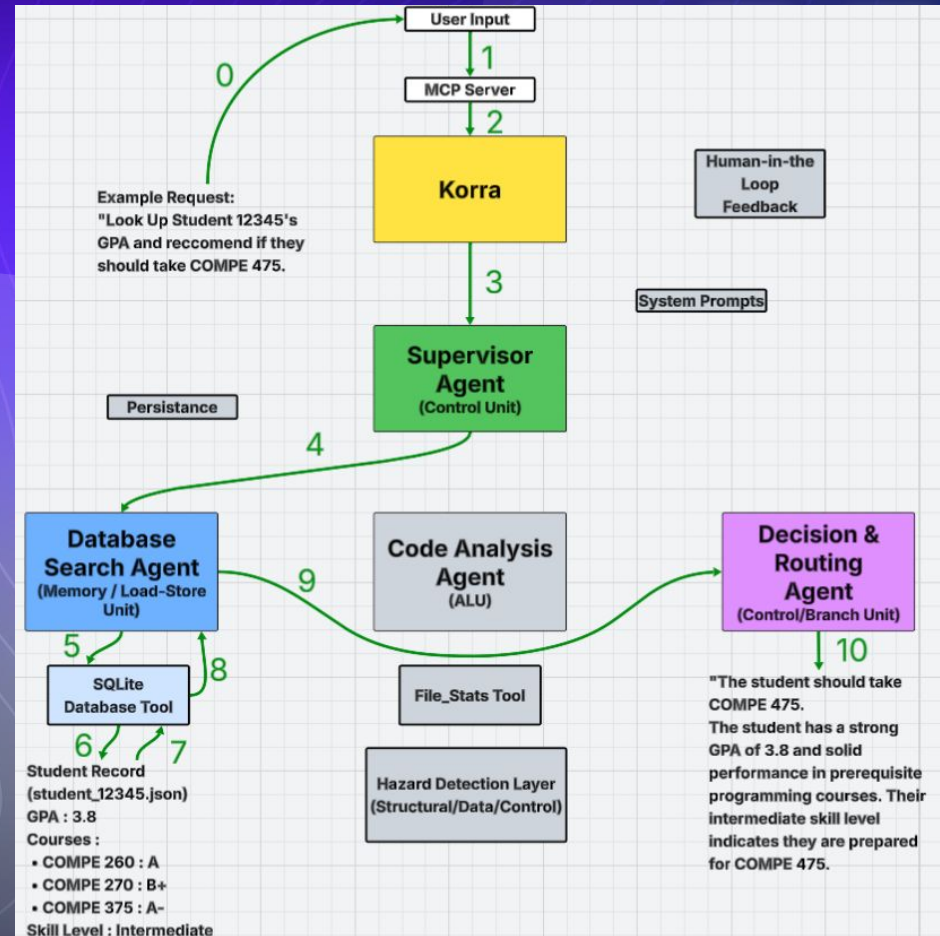
- **Module 06:** Developed a C-Based File Analysis Tool and Integrated it with Python
- **Module 07:** Added tool routing, LangGraph integration, and Docker Deployment
- **Module 08:** Implemented MCP Server and web-based interface
- **Module 09:** Added Persistence streaming, and SQLite Database Support

What We Built

- **Module 10:** Added Human-In-The-Loop Interrupt Handling and Approval Workflows
- **Module 11-12:** Implemented System Prompts to Control Agent Behavior and Execution Logic
- **Module 13:** Developed Specialized Agents for Database Actions, Code Analysis, and Decision-Making
- **Module 14:** Integrated a Supervisor to Coordinate Agents and Enable Pipeline Execution
- **Module 15:** Implemented Hazard Detection Including Structural, Data, and Control Hazards

How Korra Works

- Request enters through the UI and is passed to the supervisor
- Supervisor routes the request through a pipeline of specialized agents where data is processed between stages
- Final response is generated and returned to the user



COMPE 475 Skills Applied

- **C Programming:** Custom tools and low-level systems integration
- **Interfacing and Communication:** MCP Servers, APIs, and user interface
- **Memory-Systems:** Persistence, streaming, and database storage
- **Assembly-Level Organization:** Supervisor routing and instruction execution flow

COMPE 475 Skills Applied

- **Assembly Programming:** System prompts controlling agent behavior
- **Functional Organization:** Specialized agents acting as CPU functional units
- **Pipelining:** Multi-Agent workflow and staged execution
- **Hazard Detection:** Structural, data, and control hazard handling



What I Learned

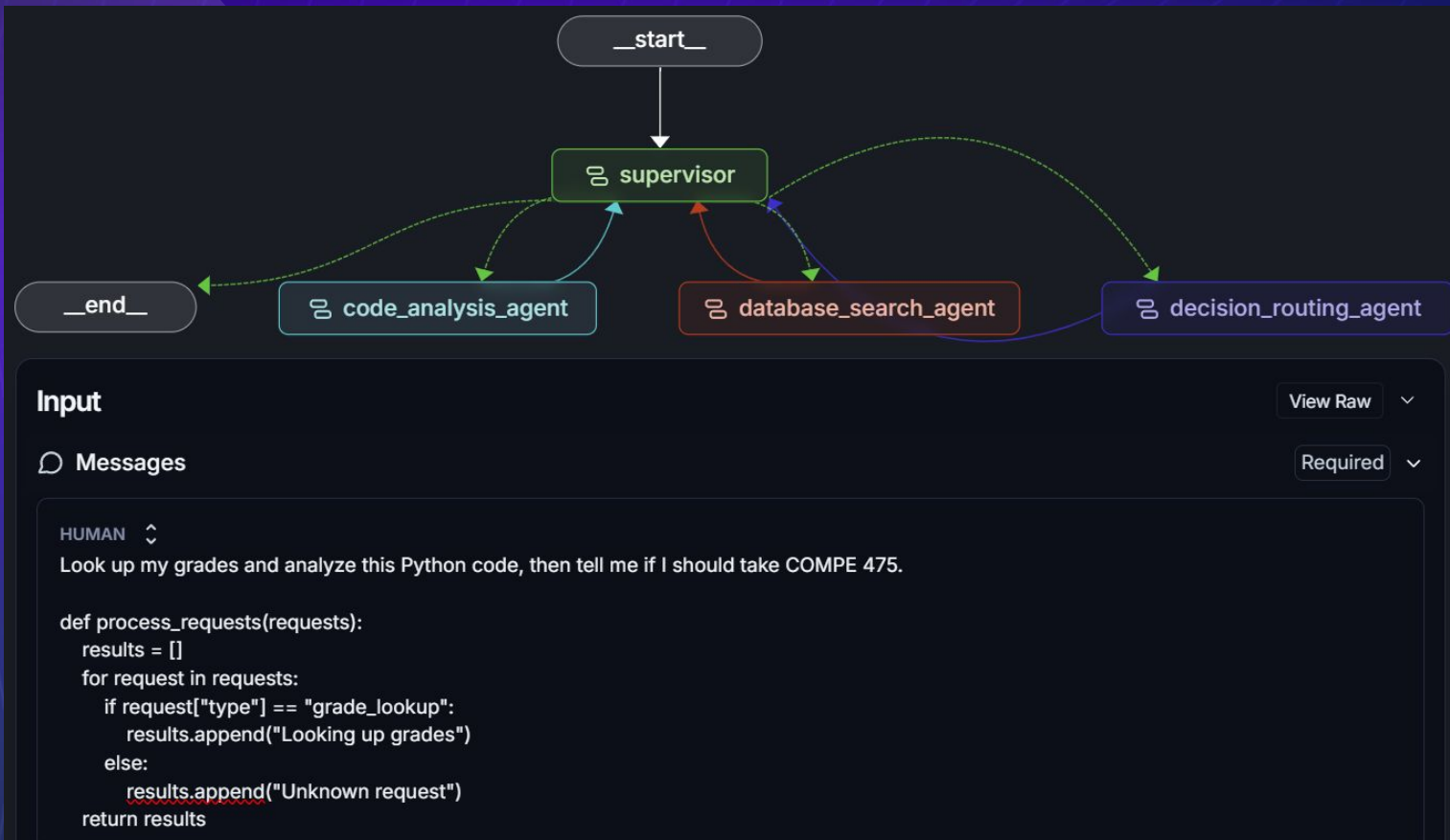
- How to build and coordinate multi-agent systems using LangGraph
- Developed a better understanding of pipelining, specialization, and hazard resolution
- Deepened understanding of CPU concepts by translating them into real system behavior





Agent

Demonstration



This prompt requires the supervisor to coordinate all three agents in a pipelined workflow

Supervisor coordinating execution by routing the request between agents and their tools, collecting intermediate results throughout the process.



supervisor

> **Human** Look up my grades and analyze this Python code, then tell...

> **AI** transfer_to_database_search_agent

> **Tool** transfer_to_database_search_agent

> **AI** OBSERVATION: Here are your grades and relevant informatio...

> **AI** transfer_back_to_supervisor

> **Tool** transfer_back_to_supervisor

> **AI** transfer_to_code_analysis_agent

> **Tool** transfer_to_code_analysis_agent

> **AI** 1. **Language**: Python 2. **What the code does**: The f...

> **AI** transfer_back_to_supervisor

> **Tool** transfer_back_to_supervisor

> **AI** transfer_to_decision_routing_agent

> **Tool** transfer_to_decision_routing_agent

> **AI** Decision: Should you take COMPE 475? Criteria: 1. Academ...

> **AI** transfer_back_to_supervisor

> **Tool** transfer_back_to_supervisor

> **AI** **### Summary** You requested a lookup of your grades, an ana...

Summary

You requested a lookup of your grades, an analysis of a Python code snippet, and a recommendation on whether to take COMPE 475.

Database Findings

- **GPA:** 3.8
- **Courses:**
 - COMPE 260: A
 - COMPE 270: B+
 - COMPE 375: A-
- **Skill Level:** Intermediate
- **Midterm Score:** 85

Code Analysis Findings

- The provided Python function `process_requests` is designed to handle requests but has limitations:
 - It only recognizes one request type (`"grade_lookup"`).
 - It lacks error handling for missing keys in requests.
 - It does not perform actual grade lookups.
- Suggested improvements include expanding functionality, implementing error handling, and integrating actual logic for grade retrieval.

Recommendation

Take COMPE 475. Your strong GPA and performance in related courses indicate that you are well-prepared for the challenges of this course. If it aligns with your interests and career goals, it will likely enhance your skills and knowledge in the field.

Final response combines outputs from each agent into a single message to the user

```
You [thread 1]: /scenario1
```

```
Starting structural hazard demo...
```

```
Request A and Request B will both target database_search_agent.
```

```
[STRUCTURAL HAZARD] Worker database_search_agent busy. Request req_c3cec8f8 waiting (blocked by Request req_6b9ccfb2)
```

```
18:16:00 [INFO] Router saw tool call: 'save_file_to_database'
```

```
18:16:00 [INFO] No tool calls in last message; ending turn.
```

```
[RESUME] Request req_c3cec8f8 now running on database_search_agent
```

```
--- Final result for thread A ---
```

```
The operation has been completed successfully. If you have any further requests, feel free to ask!
```

```
18:16:04 [INFO] Router saw tool call: 'save_file_to_database'
```

```
18:16:05 [INFO] No tool calls in last message; ending turn.
```

```
--- Final result for thread B ---
```

```
The operation to save the note has been completed successfully. If you have any further requests, feel free to ask!
```

```
Structural hazard demo complete.
```

If multiple database requests are sent at the same time, we can see how the system detects a structural hazard and resolves it by queuing requests and resuming execution when the resources become available

Challenges and Solutions

- **Challenge:** Writing the supervisor system prompt in a manner that properly coordinated the pipelined agentic flow
 - Some requests did not activate all agents that the prompt required
 - Supervisor agent would sometimes attempt to answer the prompt without routing to agents at all
- **Solution:** Implemented clear routing rules, agent roles, keyword detection and a required execution order
- **Learned:** How to effectively write clear system prompts for proper workflow in multi-agent systems

Future Improvements

- Add more specialized agents to improve system modularity
- Implement branch prediction to improve control hazard handling
- Explore out-of-order execution for more flexible task scheduling
- Implement multithreading to allow concurrent request processing





Thank You!

Do you have any questions?
dlopez5788@sdsu.edu